Madhura Jangale | D20A | Rollno.26

Experiment no. 1

Aim: Exp 1 Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Theory:

Merkle Tree and Cryptographic Hash Functions in Blockchain

Merkle Tree

A **Merkle Tree** (also called a **Hash Tree**) is a **binary tree data structure** in which:

- Every **leaf node** contains the cryptographic hash of data (e.g., transactions)
- Every **non-leaf (parent) node** contains the hash of its child nodes
- The **topmost node** is called the **Merkle Root**

Merkle Trees are widely used in **blockchain systems** to ensure **data integrity**, **security**, and **efficient verification**.

Structure of a Merkle Tree

A Merkle Tree consists of the following components:

a) Leaf Nodes

- Represent individual data blocks or transactions
- Each leaf stores the **hash of a transaction**

Example:
H1 = Hash(Tx1)
H2 = Hash(Tx2)

b) Internal (Parent) Nodes

- Generated by hashing the concatenation of two child hashes

Example:
H12 = Hash(H1 + H2)

c) Merkle Root

- The single hash at the top of the tree
- Represents **all transactions in the block**

- Stored in the **block header**

d) Tree Properties

- Usually a **binary tree**
- If the number of transactions is **odd**, the last hash is **duplicated**
- Height of the tree = $\log_2(n)$

Merkle Trees follow specific rules to maintain consistency and security:

1. **Hash Rule**
   Every node stores a cryptographic hash.
2. **Pairing Rule**
   Hashes are always combined in **pairs**.
3. **Duplication Rule**
   If there is an **odd number of hashes**, the last hash is duplicated.
4. **Root Rule**
   Only **one hash (Merkle Root)** exists at the top.
5. **Integrity Rule**
   Any change in transaction data changes the Merkle Root.

Working of a Merkle Tree

Step-by-Step Process

1. **Transaction Collection**
   - Transactions are collected into a block.
2. **Leaf Hashing**
   - Each transaction is hashed using a cryptographic hash function (e.g., SHA-256).
3. **Pairwise Hashing**
   - Hashes are paired and concatenated.
   - The concatenated values are hashed again.
4. **Recursive Hashing**
   - The process repeats until only one hash remains.
5. **Merkle Root Formation**
   - The final hash is called the **Merkle Root**.
   - It uniquely represents all transactions.

Verification

- To verify a transaction, only a **Merkle Path** ($\log_2(n)$ hashes) is required.
- This allows **efficient and fast verification**.

Benefits of Merkle Trees

1. Data Integrity

- Any tampering with a transaction changes the Merkle Root.

2. Efficient Verification

- Verifying a transaction requires only a few hashes.

3. Reduced Storage

- Only the Merkle Root needs to be stored in the block header.

4. Scalability

- Works efficiently with millions of transactions.

5. Security

- Resistant to data manipulation and fraud.

Uses of Merkle Trees

- Blockchain transaction verification
- Secure data synchronization
- Distributed systems
- Version control systems (e.g., Git)
- File integrity checking

Use Cases of Merkle Trees

a) Blockchain (Bitcoin, Ethereum)

- Stores transaction hashes
- Enables **Simplified Payment Verification (SPV)**

b) Cryptocurrency Wallets

- Lightweight clients verify transactions without downloading full blocks

c) Distributed Databases

- Ensures consistency across replicas

d) Peer-to-Peer Networks

- Efficient verification of large data sets

e) File Systems

- Detects file tampering or corruption

Cryptographic Hash Functions in Blockchain

Definition

A **cryptographic hash function** converts input data of any size into a **fixed-length hash value**.

Properties of Cryptographic Hash Functions

1. **Deterministic**
    - Same input → same output
2. **Fixed Output Length**
    - SHA-256 produces a 256-bit hash
3. **Pre-Image Resistance**
    - Cannot determine input from hash
4. **Second Pre-Image Resistance**
    - Cannot find another input with the same hash
5. **Collision Resistance**
    - Extremely difficult to find two inputs with the same hash
6. **Avalanche Effect**
    - Small change in input produces a drastic change in output

10. Common Hash Functions Used in Blockchain

Hash Function	Usage
SHA-256	Bitcoin mining, Merkle trees
Keccak-256	Ethereum
RIPEMD-160	Bitcoin addresses
SHA-3	Modern blockchain systems

11. Role of Hash Functions in Blockchain

a) Block Hashing

- Each block contains the hash of the previous block

b) Proof-of-Work

- Mining requires finding a hash below a target value

c) Transaction Integrity

- Ensures transactions cannot be altered

d) Merkle Tree Construction

- Used to generate leaf and parent hashes

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib

def sha256_hash(data):
    """
    Generates SHA-256 hash for the given input string.
    """
    sha = hashlib.sha256()
    sha.update(data.encode('utf-8'))
    return sha.hexdigest()

# User input
message = input("Enter a string to hash using SHA-256: ")

# Generate hash
hash_value = sha256_hash(message)

print("\nSHA-256 Hash:")
print(hash_value)
```

```
Enter a string to hash using SHA-256: madhura

SHA-256 Hash:
48df10eedc5e0081c3f7cb0036c758301a7e635efabc259b120047e5cba8b849
```

2.Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process

```
import hashlib
def sha256_hash(data):
    """
    Returns SHA-256 hash of the given data
    """
    return hashlib.sha256(data.encode('utf-8')).hexdigest()
# User input
message = input("Enter the data to be mined: ")
# Difficulty level (number of leading zeros)
difficulty = int(input("Enter difficulty level : "))
target_prefix = "0" * difficulty
nonce = 0
while True:
    combined_data = message + str(nonce)
    hash_result = sha256_hash(combined_data)
    if hash_result.startswith(target_prefix):
        print("Data        :", message)
        print("Nonce       :", nonce)
        print("Hash Result :", hash_result)
        break
    nonce += 1
```

```
Enter the data to be mined: madhura
Enter difficulty level : 2
Data        : madhura
Nonce       : 376
Hash Result : 001103599491fafc3578ca637c018aa20940af03f544242b55c31911991e5a
    28
```

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```
import hashlib
def proof_of_work(data, difficulty):
    nonce = 0
    target = "0" * difficulty
    while True:
        combined = data + str(nonce)
        hash_result = hashlib.sha256(combined.encode()).hexdigest()
        if hash_result.startswith(target):
            return nonce, hash_result
        nonce += 1
# Example usage
data = input("Enter data: ")
difficulty = int(input("Enter difficulty (number of leading zeros): "))

nonce, hash_result = proof_of_work(data, difficulty)
print("Nonce found:", nonce)
print("Valid Hash:", hash_result)
```

```
Enter data: h
Enter difficulty (number of leading zeros): 1
Nonce found: 26
Valid Hash: 0d368145227c320365c84b2120a560309a5fab745b05270a632d8128fb1ddf7b
```

```
Enter data: h
Enter difficulty (number of leading zeros): 3
Nonce found: 1103
Valid Hash: 000b469ca1628133ccaf405ae947e57b5191484d16da7d34996513473c82aa42

=== Code Execution Successful ===
```

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
import hashlib
def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()
def merkle_root(transactions):
    # Hash all transactions
    hashes = [sha256(tx) for tx in transactions]
```

```
    # Build the Merkle Tree
    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])  # Duplicate last hash if odd
        new_level = []
        for i in range(0, len(hashes), 2):
            combined_hash = sha256(hashes[i] + hashes[i + 1])
            new_level.append(combined_hash)
        hashes = new_level
    return hashes[0]
# Example usage
transactions = [
    "Tx1: Alice pays Bob",
    "Tx2: Bob pays Charlie",
    "Tx3: Charlie pays Dave",
    "Tx4: Dave pays Eve"
]
print("Merkle Root:", merkle_root(transactions))
```

```
Merkle Root: 8fc5b2e4190bad429227fcd3d63a35aebbbbd72ab6ee0fb6425c05b7e9d0316
    9
```